HDOC-57

Maximum Subarray Sum using Pointers and Kadane's Algorithm

Question

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named **max_subarray_sum()** using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function must:

- Find the maximum sum of any contiguous subarray in arr[] using Kadane's algorithm.
- Display the maximum subarray sum.
- If all elements are negative, display the largest (least negative) element.
- Ensure that the selected subarray contains at least one element.

Algorithm / Logic

1. Read the size n of the array and declare a variable-length array arr[n].
2. Read all n integer elements into the array.
3. Pass the base address of arr to max_subarray_sum(arr, n).
4. Initialize current_sum and max_sum with the first array element. This guarantees that at least one element is selected and correctly handles an all-negative array.
5. Traverse from the second element using pointer arithmetic. For each element, choose the larger of: the current element alone, or current_sum + current element.
6. Update max_sum whenever current_sum becomes greater than max_sum.
7. Display max_sum as the maximum contiguous subarray sum.

C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int n) {
    int current_sum = *arr;
    int max_sum = *arr;

    for (int i = 1; i < n; i++) {
        int value = *(arr + i);

        if (current_sum + value > value)
            current_sum = current_sum + value;
        else
            current_sum = value;

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    printf("Maximum subarray sum = %d\n", max_sum);
}

int main() {
    int n;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Array must contain at least one element.\n");
        return 0;
    }
```

```
    int arr[n];

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", arr + i);

    max_subarray_sum(arr, n);
    return 0;
}
```

Sample Output

```
Enter the size of the array: 8
Enter 8 elements: -2 -3 4 -1 -2 1 5 -3
Maximum subarray sum = 7
```

Test Cases

Test Case	Input Array	Expected Maximum Sum	Purpose
1	-2, -3, 4, -1, -2, 1, 5, -3	7	Mixed values; best subarray is 4, -1, -2, 1, 5
2	-8, -3, -6, -2, -5, -4	-2	All-negative array; selects least negative element
3	5	5	Single-element array
4	1, 2, 3, 4	10	All-positive array; entire array is selected
5	-5, 4, -1, 2, 1, -7, 3	6	Best subarray is 4, -1, 2, 1

Result

The program successfully finds and displays the maximum sum of a contiguous subarray using Kadane's algorithm. The array is passed using pointers, and initializing the sums with the first element correctly handles all-negative arrays while ensuring that the chosen subarray contains at least one element.